Rigid Body Simulation

Course: IG3D

Sorbonne University

Submitted By:

Name: Arafat ISLAM **ID:** 21501835

Supervised By:

Professor Dr. Kiwon Um, Telecom Paris

1. Introduction

In this lab, I implemented a basic 3D rigid body simulator in the provided codebase. The main goal was to animate a rigid box using the equations from the lecture notes. I worked mainly in `RigidSolver.hpp`, while the rendering and main update loop were already provided in `main.cpp`.

First, I completed the rigid body solver asked in the lab manual: mass, inertia tensor, force, torque, momentum update, position update and rotation update. After that, I also added my own extension so that the cube collides with the floor, bounces a few times and then stops. I also added a small correction step for the rotation matrix because with direct integration the shape can slowly become distorted.

So in the final result, the cube first moves due to gravity and the initial applied force, then rotates in the air, then hits the floor, bounces about two or three times and finally stays at rest.

2. Rigid Body Attributes

The first task was to compute the physical attributes of the box correctly. In the `Box` class, I computed the mass M, the inertia tensor in body coordinates I_0, its inverse I_0^{-1} and the initial momentum values.

For the mass, I used:

$$M = \rho\, whd$$

For the inertia tensor of a solid cuboid, I used:

$$I_0 = \begin{bmatrix} \frac{1}{12}M(h^2+d^2) & 0 & 0 \\ 0 & \frac{1}{12}M(w^2+d^2) & 0 \\ 0 & 0 & \frac{1}{12}M(w^2+h^2) \end{bmatrix}$$

This is the standard formula from the lecture notes for a rigid box around its center of mass. Then I computed the inverse of the inertia tensor and initialized the linear and angular momentum as:

$$P = MV, \qquad L = I_0\omega$$

The code I wrote for this part is:

```
M = dens * w * h * d;

I0 = Mat3f(
  (M / tReal(12)) * (h*h + d*d), tReal(0), tReal(0),
  tReal(0), (M / tReal(12)) * (w*w + d*d), tReal(0),
  tReal(0), tReal(0), (M / tReal(12)) * (w*w + h*h)
);

I0inv = I0.inverse();
Iinv = I0inv;

P = M * V;
L = I0 * omega;
```

3. Equations Used in the Solver

From the lecture notes, the rigid body state can be written as:

$$Y(t) = \begin{bmatrix} x(t) \\ R(t) \\ P(t) \\ L(t) \end{bmatrix}$$

The time derivatives are:

$$\begin{aligned} \dot{x}(t) &= v(t), \\ \dot{R}(t) &= \omega^*(t)R(t), \\ \dot{P}(t) &= F(t), \\ \dot{L}(t) &= \tau(t) \end{aligned}$$

The auxiliary equations I used are:

$$v(t) = \frac{P(t)}{M}$$

$$\omega(t) = I^{-1}(t)L(t)$$

$$I^{-1}(t) = R(t)(I_0)^{-1}R(t)^T$$

These are the main equations that I followed in the solver step. The linear part updates the translation and the angular part updates the rotation of the rigid body.

4. Force and Linear Momentum

For the force computation, I first added gravity:

$$F = Mg$$

Following the lab manual instruction to apply an instant force

$$f = \begin{bmatrix} 0.15 \\ 0.25 \\ 0.03 \end{bmatrix}$$

at the 0th vertex during the first simulation step. In my code, this is done when `_step == 0`, which is the first actual simulation update.

The code for this part is:

```
body->F = body->M * _g;
body->tau = Vec3f(0, 0, 0);

// instant force at the very first step
if(_step == 0) {
  const Vec3f f(tReal(0.15), tReal(0.25), tReal(0.03));

  const Vec3f p = body->R * body->vdata0[0] + body->X;
  const Vec3f r = p - body->X;

  body->F += f;
  body->tau += crossVec(r, f);
}
```

After computing the force, I integrated the linear momentum using:

$$P(t + \Delta t) = P(t) + \Delta t\, F(t)$$

Then I updated the velocity and position:

$$v(t) = \frac{P(t)}{M}$$

$$x(t + \Delta t) = x(t) + \Delta t\, v(t)$$

The code for this part is:

```
body->P += dt * body->F;
body->V = body->P / body->M;
body->X += dt * body->V;
```

With this part, the cube falls under gravity and also gets an additional push from the instant force.

5. Torque, Angular Momentum and Rotation

Because the external force is applied at a vertex and not at the center of mass, it also generates a torque. I used:

$$\tau = r \times f$$

where

$$r = p - x$$

is the vector from the center of mass to the contact point.

Then I updated angular momentum using:

$$L(t + \Delta t) = L(t) + \Delta t\, \tau(t)$$

From the lecture notes, the angular velocity is obtained from:

$$\omega(t) = I^{-1}(t) L(t)$$

with

$$I^{-1}(t) = R(t)(I_0)^{-1} R(t)^T$$

Finally, the rotation matrix is updated with:

$$R(t + \Delta t) = R(t) + \Delta t\, \omega^*(t) R(t)$$

The code I used is:

```
body->L += dt * body->tau;

body->Iinv = body->R *
             body->I0inv *
             body->R.transposed();

body->omega = body->Iinv * body->L;

body->R += dt *
  (crossProductMatrix(body->omega) * body->R);
```

With this, the cube rotates in the air as expected after the off-center force is applied.

6. Rotation Matrix Drift

After adding rotational motion, I saw that the box could slowly lose its rigid shape if I only integrated the rotation matrix directly. This happens because numerical integration introduces small errors and after some time the rotation matrix is no longer perfectly orthogonal.

Following lab manual to use quaternions for this issue. In my implementation, I did not add a full quaternion-based rotation update. But I added a practical correction step that re-orthonormalizes the matrix after each update. I used a simple Gram–Schmidt style idea:

- normalize the first column,
- make the second column orthogonal to the first and normalize it,
- rebuild the third column using the cross product.

The code is:

```
void orthonormalizeRotation()
{
  Vec3f c0(body->R(0,0), body->R(1,0), body->R(2,0));
  Vec3f c1(body->R(0,1), body->R(1,1), body->R(2,1));
  Vec3f c2;

  tReal n0 = normVec(c0);
  if(n0 > tReal(1e-8)) c0 /= n0;
  else c0 = Vec3f(1, 0, 0);

  c1 -= dotVec(c0, c1) * c0;

  tReal n1 = normVec(c1);
  if(n1 > tReal(1e-8)) c1 /= n1;
  else c1 = Vec3f(0, 1, 0);

  c2 = crossVec(c0, c1);

  body->R = Mat3f(c0, c1, c2);
}
```

Then I called it right after the rotation update:

```
body->R += dt * (crossProductMatrix(body->omega) * body
     ->R);
orthonormalizeRotation();
```

So I did not implement the quaternion version exactly, but I still corrected the rotation drift in a simple and practical way.

7. Extension: Floor Collision and Settling

As an extension, I added floor collision so that the cube does not just move forever. In my scene, the floor is at

$$y = -1$$

I detected collision by transforming all box vertices into world coordinates and checking the lowest one. If the lowest vertex goes below the floor, I move the body slightly upward to remove the penetration.

To compute the contact response, I used the contact point velocity:

$$v_p = v + \omega \times r$$

Then I computed the normal relative velocity with the floor:

$$v_{\text{rel}} = v_p \cdot n$$

where

$$n = (0, 1, 0)$$

is the floor normal.

If the contact point is moving into the floor, I apply an impulse in the normal direction. The impulse scalar is:

$$j = -\frac{(1+e)\, v_{\text{rel}}}{\frac{1}{M} + n \cdot ((I^{-1}(r \times n)) \times r)}$$

Then the momentum update is:

$$P \leftarrow P + J, \qquad L \leftarrow L + r \times J$$

with

$$J = jn$$

In the code, I also decreased the restitution coefficient after each bounce, so the first bounce is larger and the next ones are smaller. I also added damping to horizontal linear momentum and angular momentum. This made the cube lose energy naturally and settle down.

The main stopping condition in my extension is:

```
if(_bounceCnt >= 3) {
  setRestingState();
  return;
}
```

And in the resting state I set all motion quantities to zero:

```
body->F = Vec3f(0, 0, 0);
body->tau = Vec3f(0, 0, 0);
body->P = Vec3f(0, 0, 0);
body->L = Vec3f(0, 0, 0);
body->V = Vec3f(0, 0, 0);
body->omega = Vec3f(0, 0, 0);
```

So in the final animation, the cube falls, rotates, hits the floor, bounces two or three times and then stays at rest.

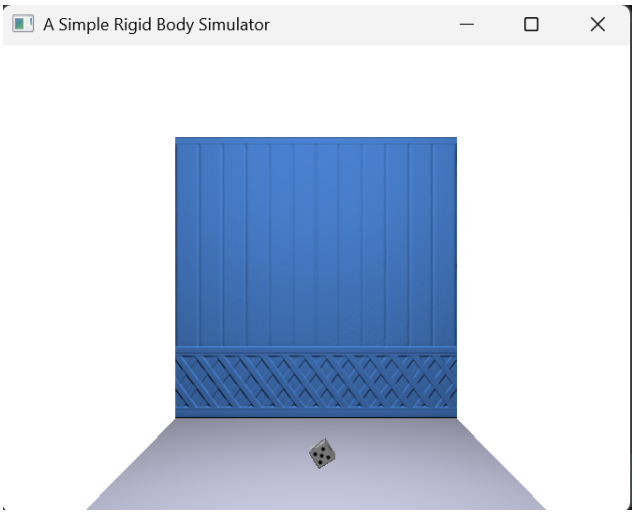

Figure 1: Final state of the cube resting on the floor after a few bounces.

8. Conclusion

In this lab, I implemented the main parts of a 3D rigid body simulator using the equations from the lecture notes. I computed the mass and inertia tensor of the box, added gravity and an initial external force and updated linear momentum, angular momentum, position and rotation over time.

The off-center force produced both translation and rotation, so the cube moved and spun in a realistic way. To keep the rotation matrix stable, I added a re-orthonormalization step. As an extra extension, I also implemented floor collision with impulse response, damping and a resting condition.

I have followed the equations from lecture notes for the basic solver and the floor bounce extension made the final animation more complete and more realistic.